

Working with Stored Procedures

In this chapter, you'll learn what stored procedures are, why they are used, and how they are used. You'll also learn the basic syntax for creating and using them.

Understanding Stored Procedures

Note

This Chapter Requires MySQL 5 Support for stored procedures was added to MySQL 5. Therefore, this chapter is applicable to MySQL 5 or later only.

Most of the SQL statements that we've used thus far are simple in that they use a single statement against one or more tables. Not all operations are that simple, though. Often, multiple statements are needed to perform a complete operation. For example, consider the following scenario:

- To process an order, checks must be made to ensure that items are in stock.
- If items are in stock, they need to be reserved so they are not sold to anyone else, and the available quantity must be reduced to reflect the correct amount in stock.
- Any items not in stock need to be ordered; this requires some interaction with the vendor.
- The customer needs to be notified about which items are in stock (and can be shipped immediately) and which are back-ordered.

This is obviously not a complete example, and it is even beyond the scope of the example tables that we have been using in this book, but it will suffice to help make a point. Performing this process requires using many MySQL statements against many tables. In addition, the exact statements that need to be performed and their order are not fixed; they can (and will) vary according to which items are in stock and which are not.

How would you write this code? You could write each of the statements individually and execute other statements conditionally, based on the result. You'd have to do this every time this processing was needed (and in every application that needed it).

Or you could create a stored procedure. A stored procedure is simply a collection of one or more MySQL statements saved for future use. You can think of stored procedures as batch files, although in truth they are more than that.

Why Use Stored Procedures

Now that you know what stored procedures are, why use them? There are many reasons, but here are the primary ones:

- They simplify complex operations (as illustrated in the previous example) by encapsulating processes into a single easy-to-use unit.
- They ensure data integrity by not requiring that a series of steps be created over and over. If all developers and applications use the same (tried and tested) stored procedure, the same code will be used by all.
An extension of this is to prevent errors. The more steps that need to be performed, the more likely it is that errors will be introduced. Preventing errors ensures data consistency.
- They simplify change management. If tables, column names, or business logic (or just about anything else) changes, only the stored procedure code needs to be updated, and no one else needs to even be aware that changes were made.
An extension of this is security. Restricting access to underlying data via stored procedures reduces the chance of data corruption (unintentional or otherwise).
- They improve performance, as they typically execute more quickly than do individual SQL statements.
- There are MySQL language elements and features that are available only within single requests. Stored procedures can use them to write code that is more powerful and flexible. (We'll see an example of this in the next chapter.)

In other words, there are three primary benefits to stored procedures: simplicity, security, and performance. Obviously, all these factors are extremely important. Before you run off and turn all your SQL code into stored procedures, though, consider the downsides:

- Stored procedures tend to be more complex to write than basic SQL statements, and writing them requires a greater degree of skill and experience.
- You might not have the security access needed to create stored procedures. Many database administrators restrict stored procedure creation rights, allowing users to execute them but not necessarily create them.

Nonetheless, stored procedures are very useful and should be used whenever possible.

Note

Can't Write Them? You Can Still Use Them MySQL distinguishes the security and access needed to write stored procedures from the security and access needed to execute them. This is a good thing; even if you can't (or don't want to) write your own stored procedures, you can still execute them, when appropriate.

Using Stored Procedures

Using stored procedures requires knowing how to execute (that is, run) them. Stored procedures are executed far more often than they are written, so we'll start there. And then we'll look at creating and working with stored procedures.

Executing Stored Procedures

MySQL refers to executing a stored procedure as *calling* the procedure, and the MySQL statement to execute a stored procedure is `CALL`. `CALL` takes the name of the stored procedure and any parameters that need to be passed to it. Take a look at this example:

► Input

```
CALL productpricing(@pricelow,  
                    @pricehigh,  
                    @priceaverage);
```

► Analysis

Here a stored procedure named `productpricing` is executed. It calculates and returns the lowest, highest, and average product prices. (And, no, you can't run this example just yet; stay tuned.)

Stored procedures might or might not display results, as you will see shortly.

Creating Stored Procedures

As already explained, writing a stored procedure is not trivial. To give you a taste for what is involved, let's look at a simple example of a stored procedure that returns the average product price. Here is the code:

► Input

```
DELIMITER //
```

```
CREATE PROCEDURE productpricing()  
BEGIN  
    SELECT Avg(prod_price) AS priceaverage  
    FROM products;  
END//
```

```
DELIMITER ;
```

► Analysis

Ignore the first and last lines for a moment; we'll come back to them shortly. The stored procedure is named `productpricing` and is defined with the statement `CREATE PROCEDURE productpricing()`. If the stored procedure accepted parameters, they would be enumerated between the (and). This stored procedure has no parameters, but the trailing () is still required. `BEGIN` and `END` statements are used to delimit the body of the stored procedure, and the body in this case is just a simple `SELECT` statement (using the `Avg()` function you learned about in Chapter 12, "Summarizing Data").

When MySQL processes this code, it creates a new stored procedure named `productpricing`. No data is returned because the code does not call the stored procedure; it simply creates the stored procedure for future use.

The DELIMITER Challenge

Back to the first and last lines in the example you just saw. As you have repeatedly seen, MySQL relies on the `;` character to terminate SQL statements. The `;` character is called the *delimiter* because it delimits (that is, defines boundaries) between SQL statements. That creates a problem in our statement. Look at this code:

```
CREATE PROCEDURE productpricing()  
BEGIN  
    SELECT Avg(prod_price) AS priceaverage  
    FROM products;  
END;
```

The closing `END;` terminates the `CREATE` statement. But look carefully, and you see that there's another `;` in there, after `products`, and that `;` terminates the statement prematurely, before the closing `END`.

The solution is to temporarily change the command-line utility delimiter, as shown here:

```
DELIMITER //
```

`DELIMITER //` instructs MySQL to use `//` instead of `;` as the new end-of-statement delimiter. And indeed, the `END` that closes the stored procedure is defined as `END //` instead of the expected `END;`. This way, the `;` within the stored procedure body remains intact and is correctly passed to the database engine. And then, things are restored back to how they were initially, like this:

```
DELIMITER ;
```

Note

Doesn't Have to Be ; Any character may be used as the delimiter except for `\`. Just be sure to use something that doesn't have a special meaning in SQL.

So how would you use this stored procedure? Like this:

► Input

```
| CALL productpricing();
```

► Output

```
|+-----+
| priceaverage |
|+-----+
|      16.133571 |
|+-----+
```

► Analysis

`CALL productpricing();` executes the just-created stored procedure and displays the returned result. Because a stored procedure is actually a type of function, `()` characters are required after the stored procedure name (even when no parameters are being passed).

Dropping Stored Procedures

After they are created, stored procedures remain on the server, ready for use, until dropped. The `DROP` command (similar to the `DROP` statement you saw in Chapter 21, “Creating and Manipulating Tables”) removes the stored procedure from the server.

To remove the stored procedure just created, use the following statement:

► Input

```
| DROP PROCEDURE productpricing;
```

► Analysis

This removes the just-created stored procedure. Notice that the trailing `()` is not used; here just the stored procedure name is specified.

Tip

Drop Only if It Exists `DROP PROCEDURE` will throw an error if the named procedure does not actually exist. To delete a procedure if it exists (and not throw an error if it does not), use `DROP PROCEDURE IF EXISTS`.

Working with Parameters

`productpricing` is a really simple stored procedure. It simply displays the results of a `SELECT` statement. Typically stored procedures do not display results; rather, they return them into variables that you specify.

New Term

Variable A named location in memory that is used to temporarily store data.

Here is an updated version of `productpricing` (but note that you won't be able to create the stored procedure again if you did not previously drop it):

► Input

```
DELIMITER //

CREATE PROCEDURE productpricing(
    OUT p1 DECIMAL(8,2),
    OUT ph DECIMAL(8,2),
    OUT pa DECIMAL(8,2)
)
BEGIN
    SELECT Min(prod_price)
    INTO p1
    FROM products;
    SELECT Max(prod_price)
    INTO ph
    FROM products;
    SELECT Avg(prod_price)
    INTO pa
    FROM products;
END//
DELIMITER ;
```

► Analysis

This stored procedure accepts three parameters: `p1` to store the lowest product price, `ph` to store the highest product price, and `pa` to store the average product price (hence the variable names). Each parameter must have its type specified; here a decimal value is used. The keyword `OUT` is used to specify that this parameter is used to send a value out of the stored procedure (back to the caller).

MySQL supports parameters of types `IN` (those passed to stored procedures), `OUT` (those passed from stored procedures, as we've used here), and `INOUT` (those used to pass parameters to and from stored procedures). The stored procedure code itself is enclosed within `BEGIN` and `END` statements, as shown earlier, and a series of `SELECT` statements are used to retrieve values that are then saved into the appropriate variables (specified with the `INTO` keyword).

Note

Parameter Datatypes The datatypes allowed in stored procedure parameters are the same as the ones used in tables. Appendix D, “MySQL Datatypes,” lists these types.

Note that a recordset is not an allowed type, and so multiple rows and columns could not be returned via a parameter. This is why three parameters (and three `SELECT` statements) are used in the previous example.

To call this updated stored procedure, three variable names must be specified, as shown here:

► Input

```
CALL productpricing(@pricelow,
                   @pricehigh,
                   @priceaverage);
```

► Analysis

As the stored procedure expects three parameters, exactly three parameters must be passed—no more and no less. Therefore, three parameters are passed to this `CALL` statement. These are the names of the three variables that the stored procedure will store the results in.

Note

Variable Names A MySQL variable name must begin with `@`.

When called, this statement does not actually display any data. Rather, it returns variables that can then be displayed (or used in other processing).

To display the retrieved average product price, you could use the following:

► Input

```
SELECT @priceaverage;
```

► Output

```
+-----+
| @priceaverage |
+-----+
| 16.133571428 |
+-----+
```

To obtain all three values, you can use the following:

► Input

```
SELECT @pricehigh, @pricelow, @priceaverage;
```

► Output

```
+-----+-----+-----+
| @pricehigh | @pricelow | @priceaverage |
+-----+-----+-----+
| 55.00      | 2.50      | 16.133571428 |
+-----+-----+-----+
```

Here is another example, this one using both IN and OUT parameters. `ordertotal` accepts an order number and returns the total for that order:

► Input

```
DELIMITER //

CREATE PROCEDURE ordertotal(
    IN onumber INT,
    OUT ototal DECIMAL(8,2)
)
BEGIN
    SELECT Sum(item_price*quantity)
    FROM orderitems
    WHERE order_num = onumber
    INTO ototal;
END //

DELIMITER ;
```

► Analysis

`onumber` is defined as IN because the order number is passed into the stored procedure. `ototal` is defined as OUT because the total is to be returned from the stored procedure. The SELECT statement uses both of these parameters, the WHERE clause uses `onumber` to select the right rows, and INTO uses `ototal` to store the calculated total.

To invoke this new stored procedure, you can use the following:

► Input

```
CALL ordertotal(20005, @total);
```

► Analysis

Two parameters must be passed to `ordertotal`; the first is the order number, and the second is the name of the variable that will contain the calculated total.

To display the total, you can then use the following:

► Input

```
SELECT @total;
```

► Output

```
+-----+
| @total |
+-----+
| 149.87 |
+-----+
```

► Analysis

`@total` has already been populated by the CALL statement to `ordertotal`, and SELECT displays the value it contains.

To obtain a display for the total of another order, you would need to call the stored procedure again and then redisplay the variable:

► Input

```
CALL ordertotal(20009, @total);
SELECT @total;
```

Building Intelligent Stored Procedures

All of the stored procedures used thus far have basically encapsulated simple MySQL `SELECT` statements. And while they are all valid examples of stored procedures, they really don't do anything more than what you could do with those statements directly. (If anything, they just make things a little more complex.) The real power of stored procedures is realized when business rules and intelligent processing are included within them.

Consider this scenario: You need to obtain order totals as before, and you also need to add sales tax to the total for some customers (perhaps the ones in your own state). Now you need to do several things:

- Obtain the total (as before).
- Conditionally add tax to the total.
- Return the total (with or without tax).

This a perfect job for a stored procedure:

► Input

```
DELIMITER //

-- Name: ordertotal
-- Parameters: onumber = order number
--             taxable = 0 if not taxable, 1 if taxable
--             ototal = order total variable

CREATE PROCEDURE ordertotal(
    IN onumber INT,
    IN taxable BOOLEAN,
    OUT ototal DECIMAL(8,2)
) COMMENT 'Obtain order total, optionally adding tax'
BEGIN

    -- Declare variable for total
    DECLARE total DECIMAL(8,2);
    -- Declare tax percentage
    DECLARE taxrate INT DEFAULT 6;

    -- Get the order total
    SELECT Sum(item_price*quantity)
    FROM orderitems
    WHERE order_num = onumber
    INTO total;
```

```

-- Is this taxable?
IF taxable THEN
    -- Yes, so add taxrate to the total
    SELECT total+(total/100*taxrate) INTO total;
END IF;

-- And finally, save to out variable
SELECT total INTO ototal;

END //

DELIMITER ;

```

► Analysis

The stored procedure has changed dramatically. First of all, comments have been added throughout (preceded by `--`). Comments are extremely important as stored procedures increase in complexity. An additional parameter has been added here: `taxable` is a `BOOLEAN` (that is true if taxable and false if not). Within the stored procedure body, two local variables are defined using `DECLARE` statements. `DECLARE` requires that a variable name and datatype be specified, and it also supports optional default values. (`taxrate` in this example is set to 6%.) The `SELECT` statement has changed so the result is stored in `total` (the local variable) instead of `ototal`. Then an `IF` statement checks to see if `taxable` is true, and if it is, another `SELECT` statement is used to add the tax to the local variable `total`. Finally, `total` (which might or might not have had tax added) is saved to `ototal` using another `SELECT` statement.

Tip

The COMMENT Keyword The stored procedure in this example includes a `COMMENT` value in the `CREATE PROCEDURE` statement. This is not required, but if it is specified, it is displayed in `SHOW PROCEDURE STATUS` results.

This is obviously a more sophisticated and powerful stored procedure. It also demonstrates a really important use for stored procedures in that it encapsulates all of the details pertaining to table and data structures and related business logic. Users could call this stored procedure to get the data they need without needing to know all the details of how the calculations work.

Now let's test the new stored procedure by using the following two statements:

► Input

```

CALL ordertotal(20005, 0, @total);
SELECT @total;

```

► Output

```

+-----+
| @total |
+-----+
| 149.87 |
+-----+

```

► Input

```
CALL ordertotal(20005, 1, @total);
SELECT @total;
```

► Output

```
+-----+
| @total |
+-----+
| 158.862200000 |
+-----+
```

► Analysis

We've added one parameter that specifies whether to calculate tax. **BOOLEAN** values may be specified as 1 for true and 0 for false. (Actually, any nonzero value is considered true, and only 0 is considered false.) By specifying 0 or 1 in the middle parameter, you can conditionally add tax to the order total.

Note

The IF Statement This example shows the basic use of the MySQL **IF** statement. **IF** also supports **ELSEIF** and **ELSE** clauses; the former also uses a **THEN** clause, and the latter does not. We'll be seeing additional uses of **IF** (as well as other flow control statements) in future chapters.

Inspecting Stored Procedures

To display the **CREATE** statement used to create a stored procedure, use the **SHOW CREATE PROCEDURE** statement:

► Input

```
SHOW CREATE PROCEDURE ordertotal;
```

To obtain a list of stored procedures, including details on when and who created them, use **SHOW PROCEDURE STATUS**.

Note

Limiting Procedure Status Results **SHOW PROCEDURE STATUS** lists all stored procedures. To restrict the output, you can use **LIKE** to specify a filter pattern, as in this example:

```
SHOW PROCEDURE STATUS LIKE 'ordertotal';
```

Summary

In this chapter, you learned what stored procedures are and why they are used. You also learned the basics of stored procedure execution and creation syntax, and you saw some of the ways stored procedures can be used. We'll continue this subject when we look at cursors in the next chapter.

Challenges

1. Create a stored procedure that accepts a customer ID and returns all orders made by that customer.
2. Different locations have different tax rates. The `ordertotal` stored procedure hard-coded the tax rate as 6%. Update that stored procedure so that it accepts the tax rate to use, if one is needed. Here's a hint: You actually don't need another parameter but could replace the `taxable` flag to accept a tax rate, with 0 meaning no tax (yay!).